Lecture 9: Experimental design and surrogate models

COMP 521 • Probability, Optimization, and Scientific Discovery • Sections 4.5–4.6

This handout is one part of the full COMP 521 notes. Section, equation, figure and listing numbers match the full notes, and references to anything outside this handout point there. The notation table is in "How to use these notes" (Lecture 1).

4.5 Optimal experimental design: what should we measure next?

A *design* ξ is something under our control: a photometric band, which stars to target, a cadence, the depth of a survey, where to run the next expensive simulation. A utility $\mathcal{U}(\xi)$ scores it, and the optimal design is $\xi^\star = \arg\max_\xi \mathcal{U}(\xi)$.

Definition 4.6 (Expected information gain)**.**

$$\mathcal{U}(\xi) = \mathbb{I}(\boldsymbol{\Theta}; \boldsymbol{Y} \mid \xi) = \mathbb{E}_{\boldsymbol{Y}\mid\xi}\Big[D_{\mathrm{KL}}\big(p(\boldsymbol{\theta} \mid \boldsymbol{Y}, \xi) \,\big\|\, p(\boldsymbol{\theta})\big)\Big] = \mathbb{E}_{\boldsymbol{\theta},\boldsymbol{Y}}\big[\log p(\boldsymbol{Y} \mid \boldsymbol{\theta}, \xi) - \log p(\boldsymbol{Y} \mid \xi)\big], \quad (36)$$

the expected Kullback–Leibler divergence from prior to posterior, which is the mutual information between parameters and data.

For densities, $D_{\mathrm{KL}}(p\|q) = \int p \log(p/q)$; it is non-negative, zero only when $p = q$ almost everywhere, and not symmetric. The (differential) entropy $\mathbb{H}[p] = -\int p \log p$ depends on units and coordinates, but the mutual information does not: it is invariant under invertible reparameterizations of $\boldsymbol{\theta}$ and of $\boldsymbol{Y}$. With natural logarithms it is measured in nats; divide by $\ln 2$ for bits. The last form of Eq. (36) is the one used for computation: it needs the likelihood and the evidence, but no posterior.

Result 4.7 (EIG in the linear-Gaussian model)**.** For $\boldsymbol{\theta} \sim \mathcal{N}(\boldsymbol{\mu}_0, \boldsymbol{\Sigma}_0)$ and $\boldsymbol{Y} = \mathbf{J}(\xi)\boldsymbol{\theta} + \boldsymbol{\epsilon}$ with $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{\Sigma}(\xi))$,

$$\mathcal{U}(\xi) = \tfrac{1}{2} \log\det\big(\mathbf{I} + \boldsymbol{\Sigma}_0\, \mathcal{I}(\xi)\big) = \tfrac{1}{2} \sum_i \log(1 + \lambda_i), \quad (37)$$

with $\mathcal{I}(\xi) = \mathbf{J}^\mathsf{T}\boldsymbol{\Sigma}^{-1}\mathbf{J}$ and λ_i the prior-whitened Fisher eigenvalues of Eq. (34). Maximizing it is Bayesian *D-optimal* design.

Equation (37) unites this section: each prior-whitened direction contributes $\frac{1}{2}\log(1 + \lambda_i)$, so a direction already well measured ($\lambda_i \gg 1$) gains only logarithmically from more of the same data, while a previously unconstrained direction gains a lot. In one dimension (the model $y = a\theta + \epsilon$ of Eq. (24)) it reduces to $\mathcal{U} = \frac{1}{2}\log(1 + a^2\tau_0^2/\sigma^2)$: with $a = 2$, $\tau_0 = 1$, $\sigma = 1$ this is $\frac{1}{2}\ln 5 \approx 0.805$ nats, and halving the noise gives $\frac{1}{2}\ln 17 \approx 1.417$ nats.

Worked example 4.8 (Precision versus complementarity)**.** Suppose we already hold $y_1 = \theta_1 + \theta_2$ with $\sigma = 0.1$ and can afford one more measurement: (a) repeat y_1 with half the error, or (b) measure the complementary $y_2 = \theta_1 - \theta_2$ with *twice* the error.

ex10_design.py -- comparing two candidate measurements by EIG

```python
"""Expected information gain (EIG) for choosing the next measurement.

Linear-Gaussian model: theta ~ N(0, Sigma0), y = J theta + noise, noise ~ N(0, Sigma).
Then EIG = 1/2 log det(I + Sigma0 J^T Sigma^-1 J)  (in nats).
We already hold y1 = theta1 + theta2 (sigma = 0.1). Which second measurement helps more?
"""
import jax
import jax.numpy as jnp

jax.config.update("jax_enable_x64", True)

Sigma0 = jnp.eye(2)


def fisher(J, sig):
    J = jnp.atleast_2d(J)
    return J.T @ jnp.diag(1.0 / jnp.asarray(sig) ** 2) @ J


def eig(F):  # EIG of all data whose total Fisher matrix is F
    return 0.5 * jnp.linalg.slogdet(jnp.eye(2) + Sigma0 @ F)[1]


F1 = fisher(jnp.array([1.0, 1.0]), [0.1])
options = {
    "(a) repeat y1, sigma = 0.05": fisher(jnp.array([1.0, 1.0]), [0.05]),
    "(b) y2 = theta1 - theta2, sigma = 0.2": fisher(jnp.array([1.0, -1.0]), [0.2]),
}
print(f"EIG of y1 alone: {eig(F1):.3f} nats")
for name, F2 in options.items():
    print(f"{name}: extra EIG = {eig(F1 + F2) - eig(F1):.3f} nats")

# Check the closed form by nested Monte Carlo in a 1-D case: y = a*theta + eps
a, s = 2.0, 1.0
k1, k2, k3 = jax.random.split(jax.random.key(5), 3)
N, M = 4000, 4000
th = jax.random.normal(k1, (N,))
y = a * th + s * jax.random.normal(k2, (N,))
th_inner = jax.random.normal(k3, (M,))
log_lik = lambda y, t: -0.5 * ((y - a * t) / s) ** 2  # constants cancel in the ratio
log_evid = jax.vmap(lambda yi: jax.scipy.special.logsumexp(log_lik(yi, th_inner))
                    - jnp.log(M))(y)
terms = log_lik(y, th) - log_evid
eig_mc, se = terms.mean(), terms.std() / jnp.sqrt(N)
print(f"1-D check: nested MC EIG = {eig_mc:.3f} +/- {se:.3f}, closed form = {0.5 * jnp.log(1 + a
    **2 / s**2):.3f}")
```

Output (executed for these notes):

```
EIG of y1 alone: 2.652 nats
(a) repeat y1, sigma = 0.05: extra EIG = 0.803 nats
(b) y2 = theta1 - theta2, sigma = 0.2: extra EIG = 1.966 nats
1-D check: nested MC EIG = 0.822 +/- 0.014, closed form = 0.805
```

The noisier but complementary measurement is worth 1.97 nats, against 0.80 nats for the more precise repeat. The last line checks Eq. (37) by nested Monte Carlo for the one-dimensional model above, estimating the evidence with an inner Monte Carlo average. The estimate, 0.822 ± 0.014, is consistent with the closed form 0.805: the difference is 1.2 standard errors. Nested estimators are

also biased: the logarithm of a noisy evidence estimate is biased low, which biases the EIG high, by an amount that shrinks as the inner sample grows (here it is far below the Monte Carlo error) [1].

In Jaxstro

Would TRGB luminosities, red-clump properties, asteroseismic frequencies, or abundance patterns constrain different combinations of uncertain Stellax physics? With a verified Stellax Jacobian for each candidate observable and realistic error budgets, Eq. (37) ranks them in minutes, before any telescope time is requested. The linear-Gaussian EIG is a first pass; near evolutionary transitions, where Eq. (12) fails, use the nested or variational estimators of [2].

4.5.1 Three different optimization questions

- **Parameter fitting:** which $\boldsymbol{\theta}$ best explains the data we have?
- **Bayesian optimization:** where should we evaluate an expensive objective next to find its minimum (for example, the stellar model that best fits a cluster)?
- **Bayesian experimental design:** which observation or simulation would teach us most about the parameters or models?

All three use surrogates, gradients and acquisition functions, but their utilities differ. Expected improvement, a Bayesian-optimization utility, is not expected information gain about stellar physics.

4.6 Surrogates, Gaussian processes, and multifidelity models

A *surrogate* (or *emulator*) $\hat{\mathcal{G}}$ is a cheap approximation of an expensive $\mathcal{G}$, *trained* on examples from $\mathcal{G}$. A Gaussian process (GP) places a prior over functions, $\mathcal{G} \sim \mathrm{GP}(m, k)$, with mean function m and kernel $k(\boldsymbol{\theta}, \boldsymbol{\theta}')$ specifying how correlated function values are at nearby inputs. Given training inputs $\mathbf{X}$ and outputs $\mathbf{f}$ (with noise variance σ_n^2), the prediction at $\boldsymbol{\theta}_*$ is Gaussian with

$$\mu_* = m(\boldsymbol{\theta}_*) + \mathbf{k}_*^\mathsf{T}(\mathbf{K} + \sigma_n^2\mathbf{I})^{-1}(\mathbf{f} - \mathbf{m}), \qquad \sigma_*^2 = k(\boldsymbol{\theta}_*, \boldsymbol{\theta}_*) - \mathbf{k}_*^\mathsf{T}(\mathbf{K} + \sigma_n^2\mathbf{I})^{-1}\mathbf{k}_*, \tag{38}$$

where $\mathbf{K}_{ij} = k(\mathbf{X}_i, \mathbf{X}_j)$ and $(\mathbf{k}_*)_i = k(\mathbf{X}_i, \boldsymbol{\theta}_*)$ [3]. For a differentiable kernel, derivative observations $\nabla\mathcal{G}(\mathbf{X}_i)$ enter by differentiating k, which is one way to put verified Stellax gradients to work: with reverse-mode AD, the gradient of a scalar output costs a small multiple of one run but carries p numbers of information.

Emulator uncertainty is not physical uncertainty

σ_* describes the surrogate's uncertainty about $\mathcal{G}$ under the kernel's assumptions, not uncertainty about the true stellar physics. If an emulator replaces Stellax inside a likelihood, add σ_*^2 to the observational variance; otherwise interpolation error masquerades as physics.

Multifidelity models combine cheap approximate and expensive accurate codes, classically as $\mathcal{G}_{\mathrm{high}}(\boldsymbol{\theta}) = a\,\mathcal{G}_{\mathrm{low}}(\boldsymbol{\theta}) + \delta(\boldsymbol{\theta})$ with a GP for δ [4]. In Jaxstro this could pair fast Startrax tracks with selected Stellax runs. Do not assume δ is small or smooth across every evolutionary phase; check it where the physics changes character. *Active learning* chooses the next training run by an acquisition function, for example expected improvement for optimization or EIG for learning; with runtimes that vary by orders of magnitude across mass, divide the utility by the cost.

Exercise for this lecture

Exercise 16, numbered as in Appendix A of the full notes. Worked solutions are in Appendix B; attempt each one before reading its solution.

16. [P] Equivalent designs. With $\theta \sim \mathcal{N}(0, 1)$, compare the EIG of: (a) one measurement $y = 3\theta + \epsilon$, $\sigma = 1$; (b) two independent measurements $y = \theta + \epsilon$, $\sigma = 1$; (c) one measurement $y = \theta + \epsilon$, $\sigma = 1/3$. Which are equivalent, and why?

References

[1] T. Rainforth, R. Cornish, H. Yang, A. Warrington, and F. Wood. On nesting Monte Carlo estimators. In *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *PMLR*, pages 4267–4276, 2018.

[2] T. Rainforth, A. Foster, D. R. Ivanova, and F. Bickford Smith. Modern Bayesian experimental design. *Statistical Science*, 39(1):100–114, 2024.

[3] C. E. Rasmussen and C. K. I. Williams. *Gaussian Processes for Machine Learning*. MIT Press, 2006.

[4] M. C. Kennedy and A. O'Hagan. Predicting the output from a complex computer code when fast approximations are available. *Biometrika*, 87(1):1–13, 2000.